Домашнее задание №4. Обеспечение отказоустойчивости средствами Apache ZooKeeper

Автор: Шайкин Андрей Сергеевич. ИСУ: 468044. Группа ИИИ-3-ИнжДан-1-3

Вариант 2 — «Программист, разработчик на Python»

Репозиторий: https://github.com/Kekendr/hh-zookeeper-failover

Итоги работы

Этап	Что сделано	Результат
1. Установка	Java 17 (Temurin 17.0.21), ZooKeeper 3.9.6 в трёх копиях, RustFS с данными ДЗ-3	`JAVA_HOME` задан, контрольные суммы архивов сверены
2. Ансамбль	три `zoo.cfg` и `myid`, запуск `zkServer.cmd`, остановка одного и двух серверов, эфемерный узел в zkCli	лидер и два последователя; без одного сервера ансамбль работает, без двух не отвечает; узел исчез после закрытия окна
3. Пауза без защиты	два координатора, лидер приостановлен на 40 с после 25 сплитов из 51	запасной стал лидером через 11,891 с и досчитал с чекпоинта; две строки «записал результат», указатель версии 2
4. Пауза с `FENCING=1`	то же с проверкой эпохи при публикации	смена лидера за 11,434 с; запись старого лидера отклонена с `BadVersionError`, указатель версии 1 от запасного
Сверка с ДЗ-3	контрольные числа и sha256 всех результатов	7 545 вакансий, 50 665 пар, sha256 `5354f749…` — совпадают во всех запусках
5. Оформление	конфигурации, журналы, выводы команд, отчёт	репозиторий `hh-zookeeper-failover`

1. Установка и стенд

Компонент	Версия и размещение
Java	Eclipse Temurin 17.0.21+9 (zip-архив), `D:\zk\jdk-17.0.21+9`, sha256 архива сверен с опубликованным
ZooKeeper	3.9.6, архив `apache-zookeeper-3.9.6-bin.tar.gz` из archive.apache.org (sha512 сверен), три копии `D:\zk\zk1`, `zk2`, `zk3`
Объектное хранилище	RustFS 1.0.1 на четырёх папках-дисках, как в ДЗ-2 и ДЗ-3; датасет заново загружен в бакет `hh-lake`, sha256 совпал с ДЗ-3
Python	3.11.9, kazoo 2.11.0 (клиент ZooKeeper), boto3 1.40.69, psutil 7.2.2
Машина	Windows 10, 6 физических / 12 логических ядер, 32 ГБ памяти

На машине уже была Java 8, а ZooKeeper 3.9 её не поддерживает, поэтому JDK 17 поставлен отдельно из zip-архива. `JAVA_HOME` задаётся в окне перед запуском: стартовый скрипт `zkEnv.cmd` берёт Java именно из этой переменной. Вариант с Docker не понадобился: три копии ZooKeeper на одной машине работают, если у каждой свои порты и своя папка данных.

```
PS C:\Users\ashai\project\inzvk\zad4\hh-zookeeper-failover> $env:JAVA_HOME='D:\zk\jdk-17.0.21+9'; & "$env:JAVA_HOME\bin\java.exe" -vers
ion; 1..3 | % { "zk$_ myid=" + (Get-Content D:\zk\zk$_\data\myid) }
openjdk version "17.0.21" 2026-10-20
OpenJDK Runtime Environment Temurin-17.0.21+9 (build 17.0.21+9)
OpenJDK 64-Bit Server VM Temurin-17.0.21+9 (build 17.0.21+9, mixed mode, sharing)
zk1 myid=1
zk2 myid=2
zk3 myid=3
```

Рис. 1. `JAVA_HOME` указывает на JDK 17, версия Java и номера `myid` трёх копий

2. Ансамбль ZooKeeper

2.1. Конфигурация

У трёх файлов `conf/zoo.cfg` одинаковые строки участников ансамбля и тайминги, а различаются только папка данных и порты, потому что все серверы работают на одном адресе 127.0.0.1:

Параметр	server.1	server.2	server.3	Зачем
`dataDir`	`D:/zk/zk1/data`	`D:/zk/zk2/data`	`D:/zk/zk3/data`	снимки и журнал транзакций у каждого свои
`myid` в `dataDir`	1	2	3	по нему сервер находит свою строку `server.N`
`clientPort`	2181	2182	2183	порт клиентов
`admin.serverPort`	8081	8082	8083	встроенный HTTP-сервер, по умолчанию у всех 8080
порты кворума и выборов	2888 / 3888	2889 / 3889	2890 / 3890	заданы в строках `server.N`, общих для всех

```
tickTime=2000
initLimit=10
syncLimit=5
dataDir=D:/zk/zk1/data
clientPort=2181
clientPortAddress=127.0.0.1
admin.serverAddress=127.0.0.1
admin.serverPort=8081
4lw.commands.whitelist=srvr,stat,ruok,mntr
server.1=127.0.0.1:2888:3888
server.2=127.0.0.1:2889:3889
server.3=127.0.0.1:2890:3890
```

В путях стоят прямые косые черты: Java читает `zoo.cfg` как файл свойств, где обратная косая черта экранирует следующий символ. Команда `srvr` разрешена в белом списке, чтобы узнавать роль сервера. На Windows `zkServer.cmd` умеет только запускать сервер, команды `status` у него нет, поэтому роли показывает скрипт `zk_status.py`. Он отправляет `srvr` на порт каждого сервера, а затем проверяет весь ансамбль: открывает сессию, создаёт, читает и удаляет тестовый узел.

2.2. Запуск и роли

Каждый сервер запущен штатной командой `zkServer.cmd` в своём окне. Серверы 1 и 2 договорились первыми: лидером стал сервер 2, у которого больше номер (при равных данных выигрывает больший `myid`). Сервер 3 запущен на 14 с позже и присоединился последователем: лидер передал ему недостающие транзакции (`Sending DIFF … for peer sid: 3`).

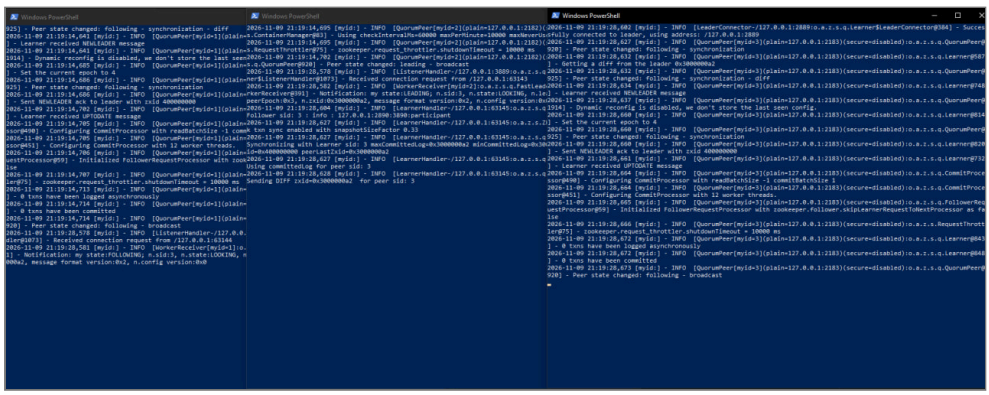


Рис. 2. Три сервера: server.1 и server.3 — `following`, server.2 — `leading` (4-я строка окна)

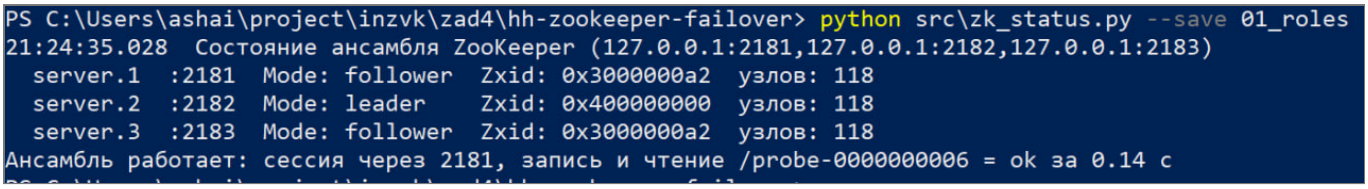


Рис. 3. Роли серверов: server.2 — leader, server.1 и server.3 — follower; запись и чтение через ансамбль проходят

2.3. Потеря одного и двух серверов

Действие	Состояние серверов	Ансамбль
все три работают (рис. 3)	server.2 — leader, Zxid `0x400000000`	работает
остановлен лидер server.2 (рис. 4)	новым лидером выбран server.3, Zxid `0x500000000`	работает: сессия через 2181, запись прошла за 2,14 с
остановлен ещё server.1 (рис. 5)	server.3 запущен, но не обслуживает запросы	не отвечает: сессия не открылась за 8 с
оба запущены снова (рис. 6)	server.3 — leader, Zxid `0x600000000`, server.1 и server.2 — follower	работает

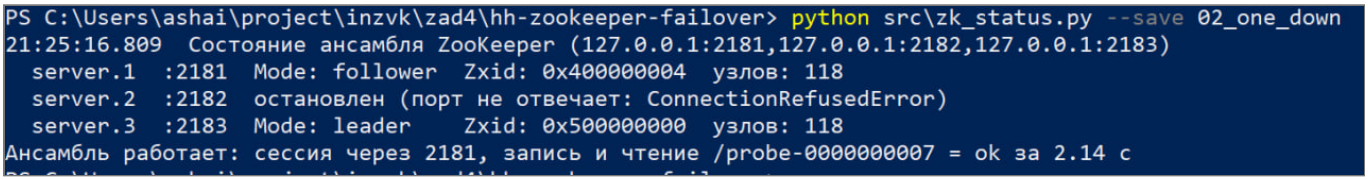


Рис. 4. Без лидера server.2: оставшиеся два сервера выбрали лидером server.3, ансамбль принимает запись

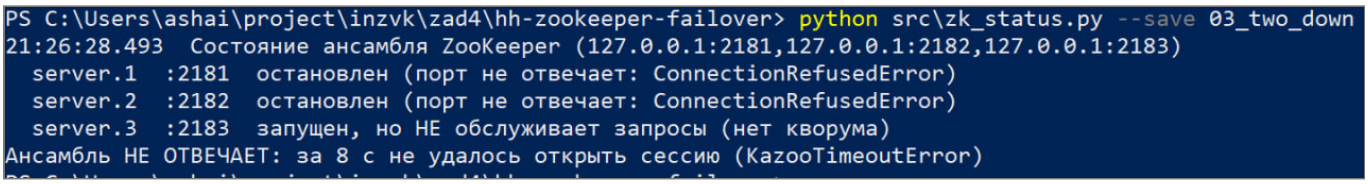


Рис. 5. Без двух серверов: server.3 жив, но не обслуживает запросы, клиент не может открыть сессию

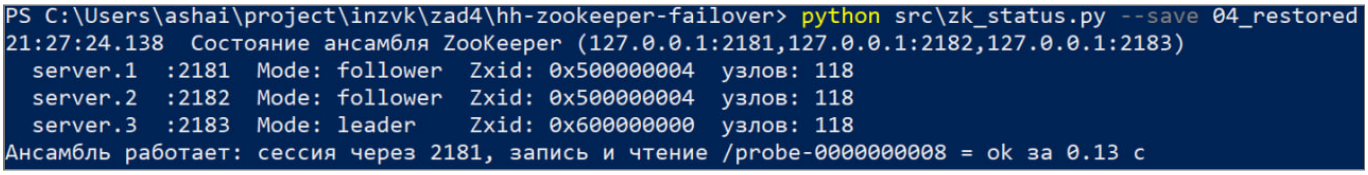


Рис. 6. Оба сервера запущены снова: ансамбль из трёх, лидер server.3

Почему так. Ансамбль работает, пока живо большинство серверов — кворум, для трёх это два. После остановки лидера два оставшихся сервера составили кворум и за пару секунд выбрали нового лидера. Поэтому первая запись на рис. 4 заняла 2,14 с вместо обычных 0,1 с: клиент ждал конца выборов. Один сервер из трёх кворумом не является. Он не принимает ни записи, ни чтения и сам переходит в состояние, в котором не обслуживает клиентов, а не работает «как получится». Это защищает от ситуации, когда отрезанный сервер отдаёт устаревшие данные.

Старшие 32 бита Zxid — это эпоха лидера ZooKeeper, она растёт при каждых выборах: `0x4…` → `0x5…` после остановки лидера → `0x6…` после возвращения серверов. Каждый новый лидер ансамбля начинает новую эпоху, и транзакции старой эпохи уже не принимаются. На этой же идее построена защита координатора в разделе 5.

2.4. Эфемерный узел

Во втором окне запущен zkCli с тайм-аутом сессии 10 с (`-timeout 10000`). В нём создан эфемерный узел командой `create -e /eph_demo shaikin-468044`, `stat` показывает `ephemeralOwner = 0x30191d4e7ff0000` — номер сессии владельца. Скрипт из первого окна видит тот же узел и ту же сессию. Старший байт номера сессии равен 3: сессию создал server.3, к нему zkCli и подключился.

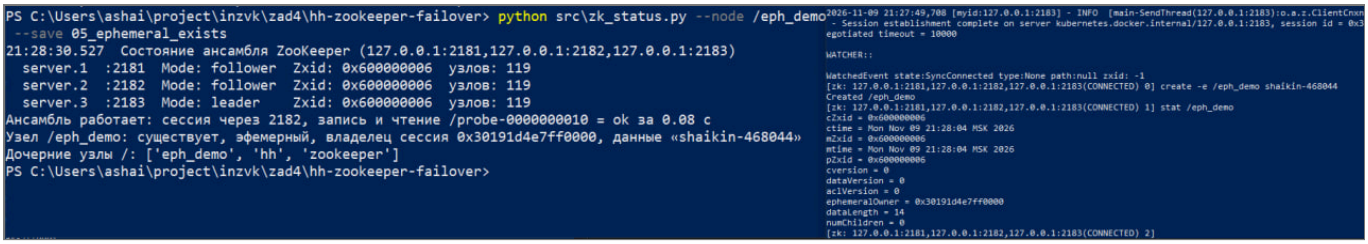


Рис. 7. Справа zkCli: создание эфемерного узла и `stat`; слева проверка из первого окна — узел эфемерный, владелец — сессия 0x30191d4e7ff0000

После закрытия окна zkCli процесс клиента завершился, не закрыв сессию, и перестал присылать пульс. Сервер выждал тайм-аут, закрыл сессию и удалил все её эфемерные узлы. Проверка через 15 с после закрытия окна показывает, что узла нет. На этом механизме держатся выборы лидера координаторов: очередь на лидерство состоит из эфемерных узлов, и узел упавшего или замершего лидера удаляется сам.

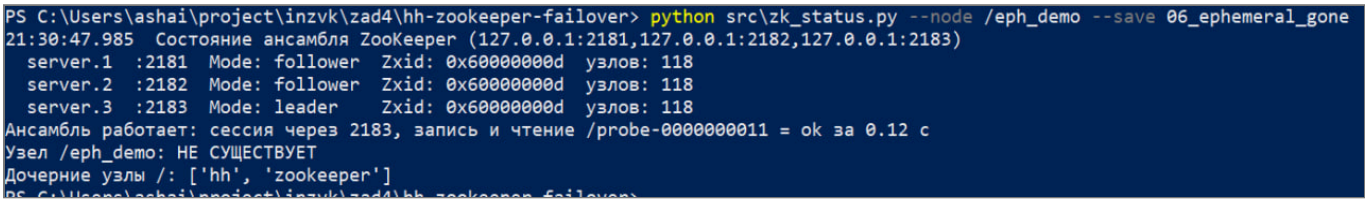


Рис. 8. После закрытия окна zkCli узла `/eph_demo` больше нет

3. Координатор расчёта

3.1. Что считается

Координатор выполняет расчёт из ДЗ-3 для варианта 2: частоты навыков, пар и троек навыков в вакансиях роли «Программист, разработчик», где python есть в навыках или в названии. Файл `it_vacancies_2024.csv` читается из бакета `hh-lake` частями (сплитами) через S3 Range-запросы. Функции чтения сплита, условие варианта, нормализация навыков и контрольная сумма sha256 перенесены из `mapreduce.py` ДЗ-3 без изменения логики (`src/hh_calc.py`). Объект поделён на 51 сплит по 16 МиБ, а не на 26 по 32 МиБ, как в ДЗ-3: чем мельче сплит, тем меньше работы теряется при отказе. Контрольная сумма от разбиения не зависит.

3.2. Устройство

Узел ZooKeeper / ключ хранилища	Назначение
`/hh/<run>/election`	очередь на лидерство (рецепт Election из kazoo): каждый координатор создаёт эфемерный последовательный узел с именем и pid, лидер — владелец узла с наименьшим номером, следующий следит за предыдущим
`/hh/<run>/epoch`	эпоха лидера: каждый новый лидер записывает в узел своё имя, и номером эпохи становится версия узла (1, 2, …)
`/hh/<run>/done/NNN`	чекпоинт: постоянный узел на каждый готовый сплит, в данных — кто и в какой эпохе его посчитал
`s3://hh-lake/zk/<run>/checkpoint/split-NNN.json.gz`	частичные счётчики навыков, пар и троек сплита
`s3://hh-lake/zk/<run>/result/<имя>-epoch<N>/`	результат лидера: `k1_skills.csv`, `k2_pairs.csv`, `k3_triples.csv`, `summary.json`
`/hh/<run>/result`	указатель на действующий результат: автор, эпоха, путь в хранилище, sha256; версия узла показывает, сколько раз его записывали

Работает так. Обе копии подключаются к ансамблю с тайм-аутом сессии 10 с и встают в очередь. Первая становится лидером, увеличивает эпоху и идёт по сплитам. После каждого сплита она сохраняет частичные счётчики в хранилище и создаёт узел `done/NNN`. Вторая копия ждёт. Если сессия лидера истекает, его эфемерный узел пропадает, запасной получает уведомление и становится лидером. Он увеличивает эпоху, читает список `done`, загружает готовые сплиты из

хранилища и считает только остальные. Когда готовы все сплиты, лидер записывает файлы результата под своим именем и эпохой, а затем обновляет указатель.

Частичные счётчики хранятся в RustFS, а в ZooKeeper — только отметки о готовности. ZooKeeper держит все данные в памяти и реплицирует каждую запись на все серверы, размер узла ограничен 1 МБ. Он предназначен для небольших координационных данных, а не для результатов расчёта.

3.3. Защита по эпохе

Без защиты указатель обновляется обычной записью `set`. С `FENCING=1` — транзакцией из двух операций: проверить, что версия узла `epoch` всё ещё равна эпохе этого лидера, и только тогда записать указатель. ZooKeeper выполняет транзакцию целиком или не выполняет ничего. Если за время работы лидера выбран новый, эпоха уже другая, проверка не проходит, и указатель не меняется.

```
if self.fencing:
    def commit():
        tx = zk.transaction()
        tx.check(p.epoch, self.epoch)      # эпоха не сменилась с момента избрания
        tx.set_data(p.result, pointer)     # тогда обновить указатель
        return tx.commit()
    res = zk.retry(commit)
    if isinstance(res[0], Exception):      # BadVersionError: лидер уже сменился
        ...
else:
    version = zk.retry(zk.set, p.result, pointer).version
```

3.4. Как приостанавливается лидер

Скрипт `pause_leader.py` берёт pid лидера из первого узла очереди выборов, ждёт, пока в чекпоинте наберётся 25 сплитов из 51, и замораживает процесс через psutil. На Windows это останавливает все потоки процесса, включая поток kazoo, который шлёт пульс сессии, — так же, как долгая сборка мусора, своп или `SIGSTOP`. Скрипт следит за очередью выборов, записывает, когда лидером стал другой координатор, и через 40 с возобновляет процесс. Координатор работает в одном процессе, чтобы заморозка останавливала весь расчёт целиком.

После каждого сплита координатор делает паузу 0,3 с. Без неё один координатор считает всё за 17,5 с (контрольный запуск `run0_baseline`, sha256 тот же). Это сравнимо со временем смены лидера (около 12 с), а паузу нужно было гарантированно попасть посреди вычислений. С паузой расчёт идёт около 30 с, лидер приостанавливается на 15-й секунде, и запасному остаётся заметная половина работы. На результат пауза не влияет.

4. Этап 3: приостановка лидера без защиты

Запуск `run1_nofence`: в трёх окнах `pause_leader.py`, координатор `coord-A` и через 4 с `coord-B`, у обоих `FENCING=0`.

Время	Кто	Событие
21:37:14.198	coord-A	стал лидером, эпоха 1, начинает с нуля
21:37:18.338	coord-B	встал в очередь за coord-A
21:37:29.350	pause	ПРИОСТАНОВИЛ coord-A: в чекпоинте 25 сплитов
21:37:41.241	coord-B	стал лидером, эпоха 2; в чекпоинте готово 25, продолжает с чекпоинта
21:37:57.656	coord-B	все сплиты готовы за 16,3 с: посчитал сам 26, взял из чекпоинта 25
21:38:00.780	coord-B	записал результат: указатель версия 1, эпоха 2
21:38:09.356	pause	ВОЗОБНОВИЛ coord-A через 40,0 с
21:38:09.371	coord-A	сессия ZooKeeper истекла (LOST), затем новая сессия
21:38:13.470	coord-A	записал результат: указатель версия 2, эпоха 1

```
PS C:\Users\ashai\project\inzvk\zad4\hh-zookeeper-failover> python src\pause_leader.py --run run1_nofence
21:37:07.407 [pause] жду двух координаторов в очереди /hh/run1_nofence/election
21:37:18.454 [pause] очередь выборов: ['coord-A pid=11428', 'coord-B pid=5072']; лидер: coord-A pid=11428
21:37:18.454 [pause] жду, пока лидер посчитает 25 сплитов
21:37:29.350 [pause] ПРИОСТАНОВИЛ лидера coord-A pid=11428: в чекпоинте 25 сплитов; статус процесса stopped
21:37:41.269 [pause] лидером стал coord-B pid=5072 через 11.92 с после приостановки
21:38:09.356 [pause] ВОЗОБНОВИЛ coord-A pid=11428 через 40.0 с; очередь выборов: []
PS C:\Users\ashai\project\inzvk\zad4\hh-zookeeper-failover>
```

Рис. 9. `pause_leader.py`*: лидер coord-A приостановлен после 25 сплитов, через 11,92 с лидером стал coord-B, через 40,0 с coord-A возобновлён*

```
PS C:\Users\ashai\project\inzvk\zad4\hh-zookeeper-failover> $env:FENCING='0'; python src\coordinator.py --run run1_nofence --name coord-B
21:37:18.242 [coord-B] старт: pid=5072, запуск run1_nofence, вариант 2: Программист, разработчик на Python
21:37:18.242 [coord-B] защита по эпохе FENCING=0 (выключена); пауза после сплита 0.3 с
21:37:18.332 [coord-B] подключён к ZooKeeper (CONNECTED)
21:37:18.332 [coord-B] ансамбль 127.0.0.1:2181,127.0.0.1:2182,127.0.0.1:2183; сессия 0x10191d4e7b90002, тайм-аут 10 с
21:37:18.338 [coord-B] встаю в очередь на лидерство /hh/run1_nofence/election; сейчас в очереди: ['coord-A pid=11428']
21:37:41.241 [coord-B] стал лидером
21:37:41.314 [coord-B] эпоха лидера: 2 (версия узла /hh/run1_nofence/epoch)
21:37:41.346 [coord-B] сплитов: 51 по 16 МиБ; в чекпоинте готово: 25, продолжаю с чекпоинта
21:37:42.059 [coord-B] сплит 26/51 посчитан: вакансий варианта 157, пар 3,757; чекпоинт сохранён
21:37:42.697 [coord-B] сплит 27/51 посчитан: вакансий варианта 207, пар 3,546; чекпоинт сохранён
21:37:43.314 [coord-B] сплит 28/51 посчитан: вакансий варианта 203, пар 3,966; чекпоинт сохранён
21:37:43.981 [coord-B] сплит 29/51 посчитан: вакансий варианта 189, пар 2,767; чекпоинт сохранён
21:37:44.597 [coord-B] сплит 30/51 посчитан: вакансий варианта 171, пар 3,967; чекпоинт сохранён
21:37:45.180 [coord-B] сплит 31/51 посчитан: вакансий варианта 141, пар 3,623; чекпоинт сохранён
21:37:45.806 [coord-B] сплит 32/51 посчитан: вакансий варианта 182, пар 3,739; чекпоинт сохранён
21:37:46.397 [coord-B] сплит 33/51 посчитан: вакансий варианта 122, пар 3,045; чекпоинт сохранён
21:37:47.039 [coord-B] сплит 34/51 посчитан: вакансий варианта 128, пар 3,521; чекпоинт сохранён
21:37:47.651 [coord-B] сплит 35/51 посчитан: вакансий варианта 166, пар 3,018; чекпоинт сохранён
21:37:48.256 [coord-B] сплит 36/51 посчитан: вакансий варианта 180, пар 3,673; чекпоинт сохранён
21:37:48.848 [coord-B] сплит 37/51 посчитан: вакансий варианта 166, пар 2,918; чекпоинт сохранён
21:37:49.414 [coord-B] сплит 38/51 посчитан: вакансий варианта 124, пар 2,431; чекпоинт сохранён
21:37:50.022 [coord-B] сплит 39/51 посчитан: вакансий варианта 107, пар 2,713; чекпоинт сохранён
21:37:50.656 [coord-B] сплит 40/51 посчитан: вакансий варианта 110, пар 2,718; чекпоинт сохранён
21:37:51.289 [coord-B] сплит 41/51 посчитан: вакансий варианта 120, пар 2,566; чекпоинт сохранён
21:37:51.889 [coord-B] сплит 42/51 посчитан: вакансий варианта 143, пар 3,633; чекпоинт сохранён
21:37:52.564 [coord-B] сплит 43/51 посчитан: вакансий варианта 146, пар 3,484; чекпоинт сохранён
21:37:53.172 [coord-B] сплит 44/51 посчитан: вакансий варианта 141, пар 2,791; чекпоинт сохранён
21:37:53.766 [coord-B] сплит 45/51 посчитан: вакансий варианта 152, пар 3,233; чекпоинт сохранён
21:37:54.381 [coord-B] сплит 46/51 посчитан: вакансий варианта 138, пар 3,753; чекпоинт сохранён
21:37:54.981 [coord-B] сплит 47/51 посчитан: вакансий варианта 152, пар 4,142; чекпоинт сохранён
21:37:55.606 [coord-B] сплит 48/51 посчитан: вакансий варианта 166, пар 3,650; чекпоинт сохранён
21:37:56.189 [coord-B] сплит 49/51 посчитан: вакансий варианта 176, пар 3,552; чекпоинт сохранён
21:37:56.788 [coord-B] сплит 50/51 посчитан: вакансий варианта 166, пар 4,344; чекпоинт сохранён
21:37:57.347 [coord-B] сплит 51/51 посчитан: вакансий варианта 108, пар 2,830; чекпоинт сохранён
21:37:57.656 [coord-B] все сплиты готовы за 16.3 с: посчитал сам 26, взял из чекпоинта 25
21:37:58.161 [coord-B] вакансий варианта: 7,545; из них с навыками: 6,637; строк данных: 229,388
21:37:58.164 [coord-B] k=1: уникальных 3,052, сумма частот 48,857; топ-3: python (6508), postgresql (2138), sql (2005)
21:37:58.215 [coord-B] k=2: уникальных 50,665, сумма частот 223,836; топ-3: postgresql + python (2097), python + sql (1988), git + python (1790)
21:37:58.610 [coord-B] k=3: уникальных 356,245, сумма частот 896,960; топ-3: git + postgresql + python (742), postgresql + python + sql (727), git + linux + python (702)
21:37:58.610 [coord-B] контрольный sha256: 5354f749b1a6f90370ab5df1c7cabf0d1d134c24a8e39b8490bd8eeefa9d98a8
21:38:00.673 [coord-B] файлы результата записаны в s3://hh-lake/zk/run1_nofence/result/coord-B-epoch2/
21:38:00.674 [coord-B] публикую результат в указатель /hh/run1_nofence/result без проверки эпохи
21:38:00.780 [coord-B] записал результат: указатель /hh/run1_nofence/result, версия 1, эпоха 2
21:38:00.872 [coord-B] работа завершена, закрываю сессию и выхожу
PS C:\Users\ashai\project\inzvk\zad4\hh-zookeeper-failover>
```

Рис. 10. Запасной coord-B: ждал в очереди, стал лидером с эпохой 2, взял 25 сплитов из чекпоинта, досчитал сплиты 26–51, записал результат — указатель версии 1

```
PS C:\Users\ashai\project\inzvk\zad4\hh-zookeeper-failover> $env:FENCING='0'; python src\coordinator.py --run run1_nofence --name coord-A
21:37:14.039 [coord-A] старт: pid=11428, запуск run1_nofence, вариант 2: Программист, разработчик на Python
21:37:14.040 [coord-A] защита по эпохе FENCING=0 (выключена); пауза после сплита 0.3 с
21:37:14.077 [coord-A] подключён к ZooKeeper (CONNECTED)
21:37:14.077 [coord-A] ансамбль 127.0.0.1:2181,127.0.0.1:2182,127.0.0.1:2183; сессия 0x10191d4e7b90001, тайм-аут 10 с
21:37:14.167 [coord-A] встаю в очередь на лидерство /hh/run1_nofence/election; сейчас в очереди: никого
21:37:14.198 [coord-A] стал лидером
21:37:14.231 [coord-A] эпоха лидера: 1 (версия узла /hh/run1_nofence/epoch)
21:37:14.254 [coord-A] сплитов: 51 по 16 МиБ; в чекпоинте готово: 0, начинаю с нуля
21:37:14.631 [coord-A] сплит  1/51 посчитан: вакансий варианта 146, пар 2,736; чекпоинт сохранён
21:37:15.267 [coord-A] сплит  2/51 посчитан: вакансий варианта 121, пар 1,840; чекпоинт сохранён
21:37:15.823 [coord-A] сплит  3/51 посчитан: вакансий варианта 112, пар 2,247; чекпоинт сохранён
21:37:16.440 [coord-A] сплит  4/51 посчитан: вакансий варианта 120, пар 1,948; чекпоинт сохранён
21:37:16.990 [coord-A] сплит  5/51 посчитан: вакансий варианта 142, пар 2,660; чекпоинт сохранён
21:37:17.557 [coord-A] сплит  6/51 посчитан: вакансий варианта 121, пар 2,172; чекпоинт сохранён
21:37:18.106 [coord-A] сплит  7/51 посчитан: вакансий варианта 107, пар 2,061; чекпоинт сохранён
21:37:18.926 [coord-A] сплит  8/51 посчитан: вакансий варианта 146, пар 1,944; чекпоинт сохранён
21:37:19.522 [coord-A] сплит  9/51 посчитан: вакансий варианта 147, пар 2,795; чекпоинт сохранён
21:37:20.139 [coord-A] сплит 10/51 посчитан: вакансий варианта 178, пар 2,947; чекпоинт сохранён
21:37:20.723 [coord-A] сплит 11/51 посчитан: вакансий варианта 158, пар 2,122; чекпоинт сохранён
21:37:21.331 [coord-A] сплит 12/51 посчитан: вакансий варианта 162, пар 2,472; чекпоинт сохранён
21:37:21.940 [coord-A] сплит 13/51 посчитан: вакансий варианта 140, пар 2,694; чекпоинт сохранён
21:37:22.576 [coord-A] сплит 14/51 посчитан: вакансий варианта 144, пар 2,777; чекпоинт сохранён
21:37:23.157 [coord-A] сплит 15/51 посчитан: вакансий варианта 123, пар 2,219; чекпоинт сохранён
21:37:23.748 [coord-A] сплит 16/51 посчитан: вакансий варианта 137, пар 2,739; чекпоинт сохранён
21:37:24.373 [coord-A] сплит 17/51 посчитан: вакансий варианта 130, пар 2,775; чекпоинт сохранён
21:37:24.982 [coord-A] сплит 18/51 посчитан: вакансий варианта 135, пар 2,019; чекпоинт сохранён
21:37:25.565 [coord-A] сплит 19/51 посчитан: вакансий варианта 134, пар 1,912; чекпоинт сохранён
21:37:26.181 [coord-A] сплит 20/51 посчитан: вакансий варианта 119, пар 2,025; чекпоинт сохранён
21:37:26.806 [coord-A] сплит 21/51 посчитан: вакансий варианта 148, пар 2,851; чекпоинт сохранён
21:37:27.423 [coord-A] сплит 22/51 посчитан: вакансий варианта 187, пар 3,272; чекпоинт сохранён
21:37:28.022 [coord-A] сплит 23/51 посчитан: вакансий варианта 162, пар 2,603; чекпоинт сохранён
21:37:28.726 [coord-A] сплит 24/51 посчитан: вакансий варианта 176, пар 2,976; чекпоинт сохранён
21:37:29.297 [coord-A] сплит 25/51 посчитан: вакансий варианта 189, пар 2,492; чекпоинт сохранён
21:38:09.360 [coord-A] связь с ZooKeeper прервана (SUSPENDED)
21:38:09.371 [coord-A] сессия ZooKeeper истекла (LOST), эфемерные узлы удалены
21:38:09.508 [coord-A] подключён к ZooKeeper (CONNECTED)
21:38:10.096 [coord-A] все сплиты готовы за 55.8 с: посчитал сам 25, взял из чекпоинта 26
21:38:10.633 [coord-A] вакансий варианта: 7,545; из них с навыками: 6,637; строк данных: 229,388
21:38:10.635 [coord-A] k=1: уникальных 3,052, сумма частот 48,857; топ-3: python (6508), postgresql (2138), sql (2005)
21:38:10.688 [coord-A] k=2: уникальных 50,665, сумма частот 223,836; топ-3: postgresql + python (2097), python + sql (1988), git + python (1790)
21:38:11.096 [coord-A] k=3: уникальных 356,245, сумма частот 896,960; топ-3: git + postgresql + python (742), postgresql + python + sql (727), git + linux + python (702)
21:38:11.096 [coord-A] контрольный sha256: 5354f749b1a6f90370ab5df1c7cabf0d1d134c24a8e39b8490bd8eeefa9d98a8
21:38:13.398 [coord-A] файлы результата записаны в s3://hh-lake/zk/run1_nofence/result/coord-A-epoch1/
21:38:13.399 [coord-A] публикую результат в указатель /hh/run1_nofence/result без проверки эпохи
21:38:13.470 [coord-A] записал результат: указатель /hh/run1_nofence/result, версия 2, эпоха 1
21:38:13.589 [coord-A] работа завершена, закрываю сессию и выхожу
PS C:\Users\ashai\project\inzvk\zad4\hh-zookeeper-failover>
```

Рис. 11. Бывший лидер coord-A: посчитал сплиты 1–25 и замер; после возобновления узнал об истёкшей сессии, взял остальные 26 сплитов из чекпоинта и всё равно записал результат — указатель версии 2

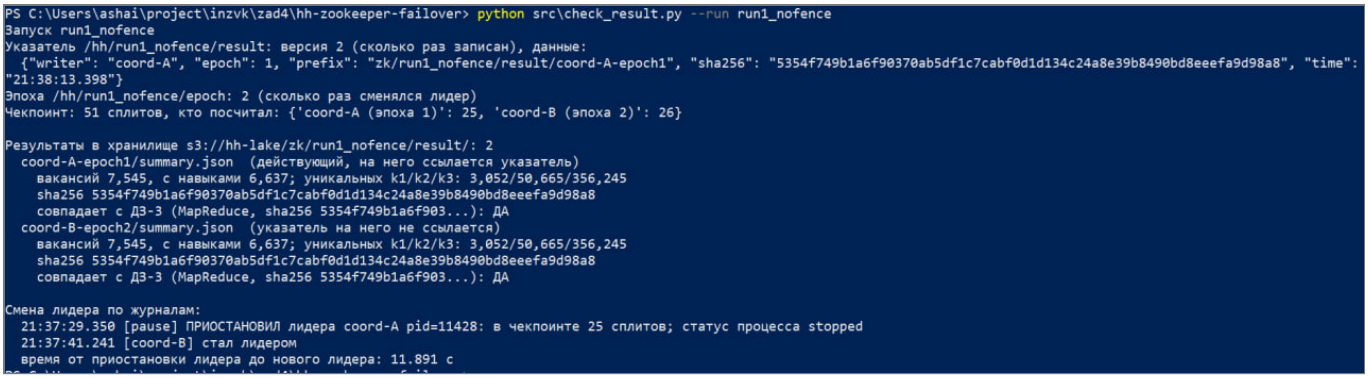


Рис. 12. Итог запуска: указатель записан дважды (версия 2) и указывает на результат coord-A с эпохой 1; оба результата совпадают с ДЗ-3; смена лидера — 11,891 с

Что произошло. Пока coord-A был заморожен, ZooKeeper закрыл его сессию и удалил его узел в очереди, и coord-B законно стал лидером. Сам coord-A об этом не знал. Проснувшись, он продолжил с того места, где остановился. Об истёкшей сессии он узнал через 15 мс после возобновления, но код публикации этого не учитывает. В итоге coord-A записал указатель через 32 с после того, как перестал быть лидером. В журнале две строки «записал результат», версия указателя 2, и действующим стал результат старой эпохи 1 поверх результата эпохи 2.

Здесь вреда не видно, потому что оба результата одинаковы: coord-A после пробуждения взял готовые сплиты coord-B из чекпоинта. Но это совпадение. Если бы старый лидер держал в памяти устаревшее состояние (другую версию входных данных, недосчитанные сплиты), его запись незаметно заменила бы правильный результат нового лидера. Читатели увидели бы откат с эпохи 2 на эпоху 1.

5. Этап 4: приостановка лидера с защитой по эпохе

Запуск `run2_fencing`: то же самое с `FENCING=1` в окнах обоих координаторов.

Время	Кто	Событие
21:40:46.760	coord-A	стал лидером, эпоха 1
21:41:01.808	pause	ПРИОСТАНОВИЛ coord-A: в чекпоинте 25 сплитов
21:41:13.242	coord-B	стал лидером, эпоха 2; продолжает с чекпоинта
21:41:29.783	coord-B	все сплиты готовы за 16,4 с: посчитал сам 26, взял из чекпоинта 25
21:41:32.843	coord-B	записал результат: указатель версия 1, эпоха 2
21:41:41.831	pause	ВОЗОБНОВИЛ coord-A через 40,0 с
21:41:45.842	coord-A	BadVersionError: запись результата отклонена, моя эпоха 1, текущая эпоха 2; указатель не изменён

```
PS C:\Users\ashai\project\inzvk\zad4\hh-zookeeper-failover> python src\pause_leader.py --run run2_fencing
21:40:42.527 [pause] жду двух координаторов в очереди /hh/run2_fencing/election
21:40:51.073 [pause] очередь выборов: ['coord-A pid=22092', 'coord-B pid=436']; лидер: coord-A pid=22092
21:40:51.073 [pause] жду, пока лидер посчитает 25 сплитов
21:41:01.808 [pause] ПРИОСТАНОВИЛ лидера coord-A pid=22092: в чекпоинте 25 сплитов; статус процесса stopped
21:41:13.284 [pause] лидером стал coord-B pid=436 через 11.48 с после приостановки
21:41:41.831 [pause] ВОЗОБНОВИЛ coord-A pid=22092 через 40.0 с; очередь выборов: []
```

Рис. 13. `pause_leader.py` во втором запуске: приостановка после 25 сплитов, смена лидера через 11,48 с, возобновление через 40,0 с

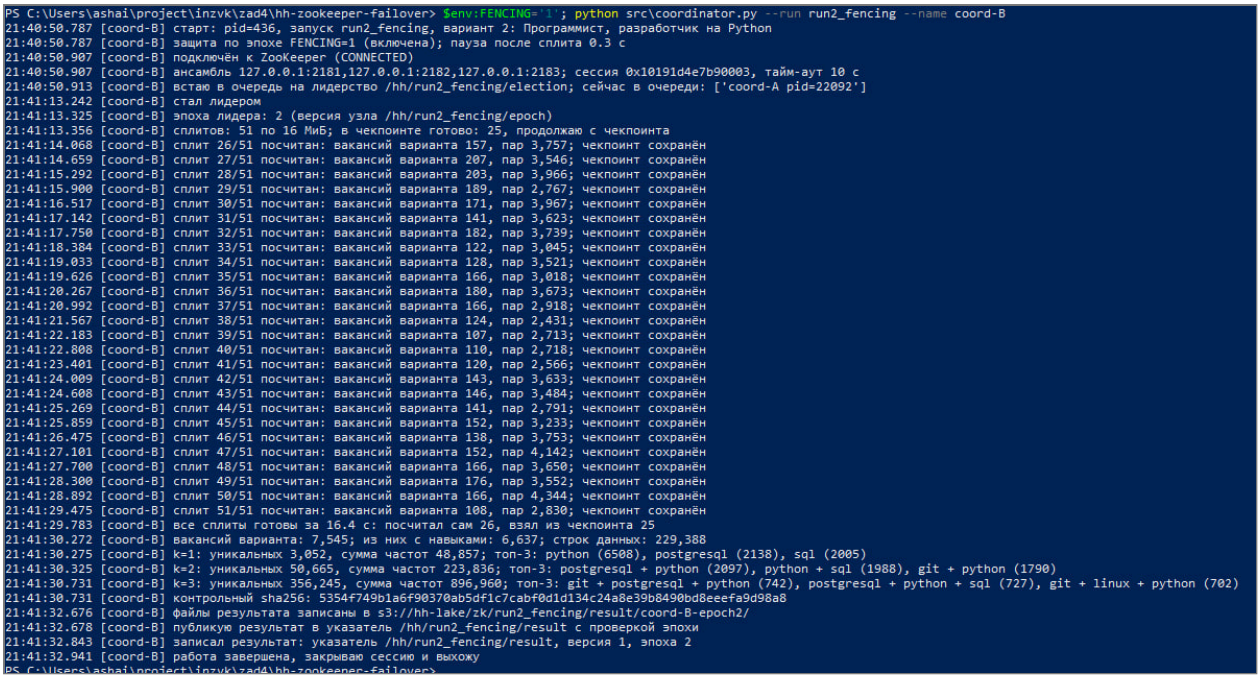


Рис. 14. Запасной coord-B с `FENCING=1`: эпоха 2, продолжение с чекпоинта, публикация с проверкой эпохи прошла — указатель версии 1

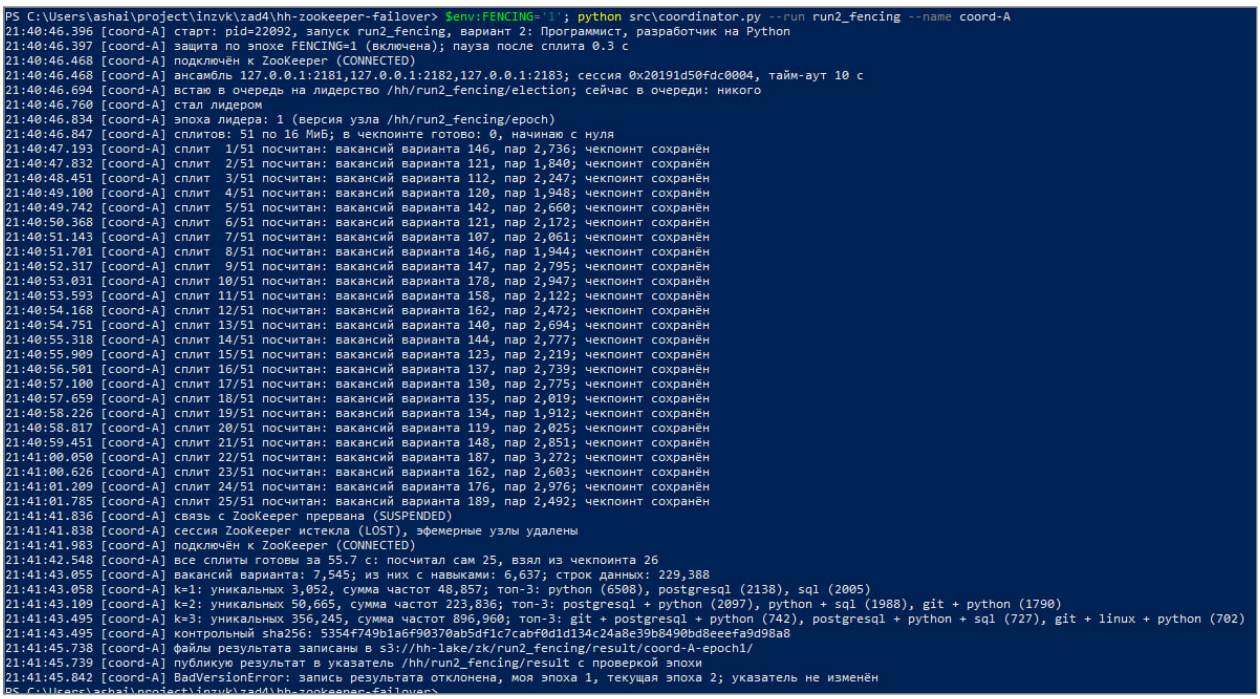


Рис. 15. Бывший лидер coord-A с `FENCING=1`: после возобновления досчитал и попытался опубликовать результат, запись отклонена с BadVersionError

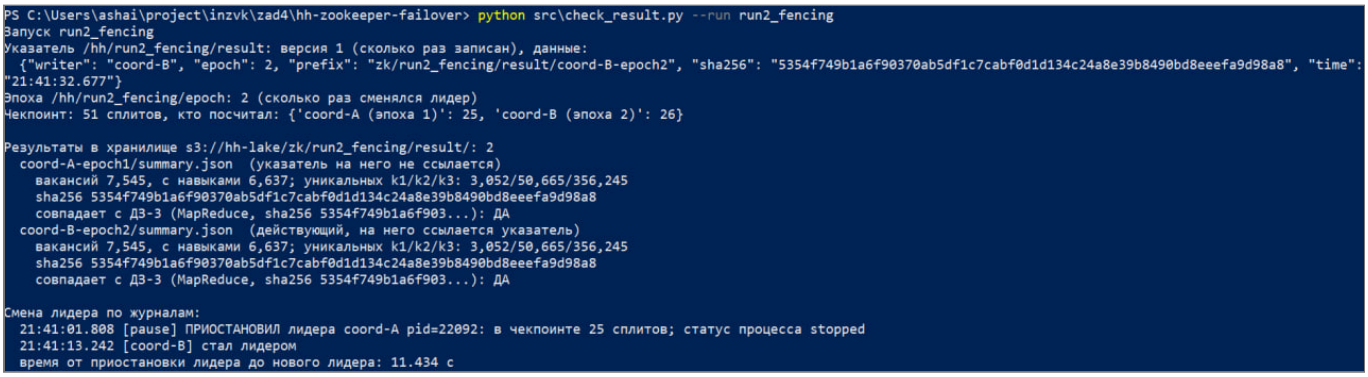


Рис. 16. Итог запуска с защитой: указатель записан один раз (версия 1) и указывает на результат coord-B с эпохой 2; результаты совпадают с ДЗ-3; смена лидера — 11,434 с

Что изменилось. До публикации coord-A повёл себя так же, как в этапе 3: проснулся, досчитал по чекпоинту и записал свои файлы в хранилище. Но обновить указатель он не смог. Его транзакция проверяла, что эпоха равна 1, а coord-B, став лидером, уже сделал её равной 2. Транзакция отклонена целиком, указатель остался от coord-B, и запись в нём одна. Файлы coord-A лежат в хранилище под отдельным путём `coord-A-epoch1`, но на них никто не ссылается: читатели находят результат только через указатель. Такие файлы можно удалять при очистке. Это обычная схема: данные пишутся в уникальное место, а атомарная смена указателя служит точкой фиксации.

6. Время смены лидера

Запуск	Приостановка	Новый лидер	По журналам, с	По pause_leader, с
этап 3, `run1_nofence`	21:37:29.350	21:37:41.241	11,891	11,92
этап 4, `run2_fencing`	21:41:01.808	21:41:13.242	11,434	11,48

Время считается от строки «ПРИОСТАНОВИЛ» в журнале `pause_leader.log` до строки «стал лидером» в журнале запасного координатора. Все процессы работают на одной машине, часы общие. `check_result.py` вычисляет это время по журналам (рис. 12 и 16), и оно согласуется с собственным замером `pause_leader.py`, который видит смену через очередь выборов на несколько десятков миллисекунд позже.

Из чего складываются 11,4–11,9 с. Основная часть — тайм-аут сессии 10 с: пока он не истёк, ZooKeeper не может отличить замерший процесс от медленного. Ещё до 2 с добавляет сам сервер: он проверяет истечение сессий не непрерывно, а пачками раз в `tickTime` = 2 с, и время истечения округляется вверх до ближайшей такой проверки. Поэтому смена лидера при тайм-ауте 10 с занимает от 10 до 12 с. Удаление узла, уведомление запасного и захват лидерства занимают десятки миллисекунд. Меньший тайм-аут ускорит смену, но тогда лидер будет терять лидерство при коротких задержках сети или сборке мусора. ZooKeeper допускает тайм-аут от 2·tickTime = 4 с до 20·tickTime = 40 с.

7. Сверка с ДЗ-3

Показатель	ДЗ-3 (MapReduce)	run0_baseline	run1, coord-B / coord-A	run2, coord-B / coord-A
вакансий варианта	7 545	7 545	7 545 / 7 545	7 545 / 7 545
из них с навыками	6 637	6 637	6 637 / 6 637	6 637 / 6 637
уникальных навыков	3 052	3 052	3 052 / 3 052	3 052 / 3 052
уникальных пар	50 665	50 665	50 665 / 50 665	50 665 / 50 665
уникальных троек	356 245	356 245	356 245 / 356 245	356 245 / 356 245
сумма частот пар	223 836	223 836	223 836 / 223 836	223 836 / 223 836
самая частая пара	postgresql + python (2097) во всех результатах			
sha256 навыков, пар и троек	`5354f749b1a6f90370ab5df1c7cabf0d1d134c24a8e39b8490bd8eeefa9d98a8` во всех результатах			

sha256 считается по всем 409 962 сочетаниям с их частотами так же, как в ДЗ-3, поэтому его совпадение означает совпадение результата целиком, а не только итоговых чисел. В ДЗ-3 этот результат был подтверждён независимым подсчётом в DuckDB. Совпадение у запасного координатора показывает, что продолжение с чекпоинта ничего не теряет и не считает дважды. Сплиты 1–25 посчитал один процесс, 26–51 другой, и сумма та же, что за один проход.

8. Ответы на вопросы

8.1. Почему серверов три, а не два и не четыре

Ансамбль принимает решение — запись или выбор лидера — только когда его подтверждает большинство серверов, больше половины. Отсюда сколько серверов может отказать:

Серверов	Большинство	Переживает отказов	Комментарий
1	1	0	одна точка отказа
2	2	0	хуже одного: остановка любого из двух останавливает всё
3	2	1	минимальный отказоустойчивый ансамбль
4	3	1	столько же отказов, что у трёх, но дороже
5	3	2	следующий осмысленный размер

Два сервера. Каждой записи нужны оба, поэтому отказ любого из них останавливает ансамбль. Вероятность простоя выше, чем у одного сервера: отказать могут два. Разрешить работать одному из двух нельзя. При обрыве связи между ними каждый решил бы, что второй умер, оба стали бы лидерами и начали бы принимать разные записи (split brain). Большинство нужно как раз для того, чтобы две половины не могли работать одновременно: у двух непересекающихся групп не может быть большинства у обеих.

Три сервера. Большинство — два, поэтому один отказ переживается. Это видно в работе: без server.2 оставшиеся два выбрали лидера и приняли запись (рис. 4), а без двух серверов последний живой отказался обслуживать клиентов (рис. 5).

Четыре сервера. Большинство — три, поэтому переживается тоже только один отказ. Четвёртый сервер не добавляет надёжности, но каждую запись теперь подтверждают три сервера вместо двух, растут трафик и задержка, а шанс потерять сразу два сервера из четырёх выше, чем из трёх. При разрыве сети пополам (2 + 2) не работает ни одна половина, а у трёх серверов одна сторона всегда остаётся с большинством. Поэтому ансамбли делают нечётными: 3 переживает один отказ, 5 — два.

8.2. Почему защиту по эпохе проверяет получатель записи, а не ZooKeeper

ZooKeeper видит только сессии клиентов. Когда coord-A замер, всё, что ZooKeeper смог сделать, — через тайм-аут закрыть его сессию и удалить его узел в очереди. Остановить сам процесс он не может, как и отозвать запись, которую тот уже собирается отправить. Сам coord-A о потере лидерства узнаёт, только когда проснётся и обратится к ZooKeeper. До этого он искренне считает себя лидером. Этап 3 это и показал: рецепт выборов отработал правильно, лидером стал coord-B, но coord-A всё равно записал результат через 32 с после смены.

Проверка «я ещё лидер?» на стороне самого координатора проблему не решает. Между проверкой и записью процесс может снова замереть — на сборку мусора, своп или из-за сети — и запись уйдёт уже устаревшей. Сравнивать эпоху нужно в тот момент, когда запись применяется, и атомарно с ней. Сделать это может только тот, кто запись принимает: он хранит данные и знает, запись какой эпохи принял последней. Получатель отклоняет всё, что пришло от эпохи старше уже виденной, и неважно, сколько запись была в пути и где её автор спал.

В нашей работе получатель — узел-указатель `result`, и проверку эпохи выполняет транзакция при его записи. То, что указатель тоже хранится в ZooKeeper, не меняет сути. Здесь ZooKeeper выступает как хранилище, принимающее запись с условием, а не как сервис выборов. Если бы результат записывался прямо в базу или в S3, проверять эпоху должны были бы они: хранить номер последней эпохи и принимать запись только с не меньшим (в S3 — условная запись по ETag). Сам ZooKeeper о таких записях ничего не знает и помешать им не может.

9. Выводы

- Ансамбль ZooKeeper из трёх серверов собран на одной машине: три копии 3.9.6 на Java 17, у каждой свои `dataDir`, `myid` и порты при общем списке участников. Роли видны: один лидер и два последователя.
- Без одного сервера ансамбль выбрал нового лидера и продолжил работу, без двух — перестал обслуживать клиентов. Каждые выборы увеличивали эпоху ZooKeeper в Zxid с `0x4` до `0x6`.
- Эфемерный узел живёт, пока жива сессия: после закрытия окна zkCli сервер дождался тайм-аута и удалил узел. На этом построены выборы лидера координаторов.
- Координатор расчёта из ДЗ-3 переживает остановку лидера. Запасной становится лидером через 11,4–11,9 с (10 с тайм-аута сессии и до 2 с на проверку истечения), берёт 25 готовых сплитов из чекпоинта и досчитывает остальные 26. Результат во всех запусках совпадает с ДЗ-3 вплоть до sha256 всех 409 962 сочетаний.

- Выборы лидера сами по себе не защищают от старого лидера, который проснулся после паузы. Без защиты он записал результат второй раз (указатель версии 2) поверх результата нового лидера. С защитой по эпохе его запись отклонена с `BadVersionError`, и указатель остался от запасного с версией 1.

Приложение. Репозиторий

https://github.com/Kekendr/hh-zookeeper-failover. Порядок запуска описан в README.

Файл	**Назначение**
`conf/zk1..zk3/zoo.cfg`	конфигурации трёх серверов ансамбля
`src/coordinator.py`	координатор: выборы лидера, чекпоинт, публикация результата, `FENCING=1`
`src/hh_calc.py`	расчёт навыков, пар и троек по сплиту, перенесён из ДЗ-3
`src/pause_leader.py`	приостановка лидера посреди расчёта и замер смены лидера
`src/zk_status.py`	роли серверов, проверка ансамбля, проверка узла
`src/check_result.py`	указатель результата, сверка с ДЗ-3, время смены лидера по журналам
`src/settings.py`	адреса ансамбля и хранилища, журналы
`reference/dz3_mapreduce_summary.json`	итог MapReduce из ДЗ-3 для сверки
`logs/run0_baseline/`, `logs/run1_nofence/`, `logs/run2_fencing/`	журналы координаторов и `pause_leader` всех запусков
`outputs/`	выводы `zk_status.py` (рис. 3–8) и `check_result.py` (рис. 12, 16) текстом